



Daffodil
International
University

Lab Report

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	25
Understanding	3					
Analysis	4					
Implementation	8					
Report Writing	10					
Total obtained mark						
Comments						

Semester: Summer-2026

Student Name: Md. Abdullah Al Noman

Student ID: 232-35-003

Batch: 41

Section: G1

Course Code: DS 332

Course Name: Introduction to

Data Science and Data Management & Analysis Lab

Course Teacher Name: Syed Shams Islam

Designation: Lecturer

Submission Date: 06 / 08 / 2026

Exploratory Data Analysis on Covid-19 Global Dataset

Project Overview

This project analyzes a worldwide COVID-19 dataset containing information about confirmed cases, deaths, recoveries, testing, population, and other statistics for different countries. The goal is to understand the data, discover useful patterns, and build a simple machine learning model to predict COVID-19 death rates.

Research Question: How did COVID-19's impact (cases, deaths, testing) vary across countries and continents, and what factors seem related to how hard a country was hit?

Objective:

- Understand the structure and quality of the dataset.
- Clean the dataset before analysis.
- Explore relationships between different COVID-19 variables.
- Create new useful features from existing data.
- Build a machine learning model to predict deaths per one million people.
- Group countries with similar COVID-19 characteristics using clustering.

Expected Outcome:

- A clean and understandable dataset.
- Important insights about COVID-19 across countries.
- A prediction model for death rates.
- Country clusters based on COVID-19 statistics.

Dataset Description

Source: The dataset contains worldwide COVID-19 statistics collected for multiple countries.

Dataset Link (Kaggle): https://www.kaggle.com/datasets/josephassaker/covid19-global-dataset?select=worldometer_coronavirus_summary_data.csv

Number of Rows and Columns:

```
df = pd.read_csv('./raw dataset/dataset.csv')
print(f"Shape: {df.shape[0]} rows x {df.shape[1]} columns")
df.head()
```

✓ 7.7s Python

Shape: 226 rows x 12 columns

	country	continent	total_confirmed	total_deaths	total_recovered	active_cases	serious_or_critical	total_cases_per_1m_population	total_deaths_per_1m_population	total_tests	total_tests_per_1m_population
0	Afghanistan	Asia	179267	7690.0	162202.0	9375.0	1124.0	4420	190.0	951337.0	
1	Albania	Europe	275574	3497.0	271826.0	251.0	2.0	95954	1218.0	1817530.0	
2	Algeria	Africa	265816	6875.0	178371.0	80570.0	6.0	5865	152.0	230861.0	
3	Andorra	Europe	42156	153.0	41021.0	982.0	14.0	543983	1974.0	249838.0	
4	Angola	Africa	99194	1900.0	97149.0	145.0	NaN	2853	55.0	1499795.0	

Rows: 1 Row per country

Columns: Multiple columns containing Covid -19 Stats

Features and their meaning:

Column	Meaning
country	Country name (identifier, not a feature for stats)
continent	Continent our main categorical grouping variable
total_confirmed	Cumulative confirmed COVID-19 cases
total_deaths	Cumulative deaths
total_recovered	Cumulative recoveries
active_cases	Cases still active at snapshot time
serious_or_critical	Currently in serious/critical condition
total_cases_per_1m_population	Cases normalized per 1 million people
total_deaths_per_1m_population	Deaths per 1 million people
total_tests	Cumulative tests administered
total_tests_per_1m_population	Tests per 1 million people
population	Country population

Target variable: We will target total_deaths_per_1m_population

Data types: Form the dataset we got:

```
# Data types - flags anything that should be null
df.dtypes

country          str
continent        str
total_confirmed  int64
total_deaths     float64
total_recovered  float64
active_cases     float64
serious_or_critical float64
total_cases_per_1m_population int64
total_deaths_per_1m_population float64
total_tests      float64
total_tests_per_1m_population float64
population       int64
dtype: object
```

Initial observations:

```
# Initial Observations
print(df.describe(include='all'))
print(df.head(5))
print(df.tail(5))
print(df.sample(5, random_state=42))
print(df.columns)
print(df.info())
• print(df.isnull().sum())
print(df.duplicated().sum())

✓ 0.0s Python
```

- 226 rows, 12 columns a small-to-medium sized dataset, easy to fully inspect by hand.
- 6 continents represented; Africa has the most countries listed (58).
- Several columns have missing values, most noticeably serious_or_critical.
- Raw count columns (total_confirmed, total_deaths, population) are heavily right-skewed a handful of large countries dominate the scale.

Data Understanding

Before cleaning anything, the dataset was checked for three classic problems: missing values, duplicates, and inconsistent/impossible values.

Dataset structure: we use `df.info()` to show the structure of dataset.

```
# Structure
df.info()

✓ 0.0s

<class 'pandas.DataFrame'>
RangeIndex: 226 entries, 0 to 225
Data columns (total 12 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   country                                   226 non-null    str
1   continent                                 226 non-null    str
2   total_confirmed                          226 non-null    int64
3   total_deaths                             218 non-null    float64
4   total_recovered                          204 non-null    float64
5   active_cases                             204 non-null    float64
6   serious_or_critical                      145 non-null    float64
7   total_cases_per_1m_population            226 non-null    int64
8   total_deaths_per_1m_population           218 non-null    float64
9   total_tests                              212 non-null    float64
10  total_tests_per_1m_population            212 non-null    float64
11  population                               226 non-null    int64
dtypes: float64(7), int64(3), str(2)
memory usage: 21.3 KB
```

Summary statistics: we use `df.describe()` to show the statistics of dataset.

```
# Summary statistics - numeric columns
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
total_confirmed	226.0	2.305651e+06	7.575510e+06	2.0	24126.00	179375.0	1090902.00	8.420947e+07
total_deaths	218.0	2.884442e+04	9.971254e+04	1.0	237.00	2251.5	14006.50	1.026646e+06
total_recovered	204.0	2.256851e+06	7.613357e+06	1.0	16193.00	137274.0	1006244.75	8.124426e+07
active_cases	204.0	6.861029e+04	2.350430e+05	0.0	239.75	1634.0	19296.75	1.938567e+06
serious_or_critical	145.0	2.694828e+02	8.831747e+02	1.0	5.00	17.0	139.00	8.318000e+03
total_cases_per_1m_population	226.0	1.481568e+05	1.552029e+05	16.0	11748.25	98271.5	255632.75	7.043020e+05
total_deaths_per_1m_population	218.0	1.157550e+03	1.212834e+03	2.0	172.50	768.0	1850.00	6.297000e+03
total_tests	212.0	2.987445e+07	1.079635e+08	5117.0	347296.75	2238917.5	12338623.50	1.016883e+09
total_tests_per_1m_population	212.0	1.944650e+06	3.318400e+06	5093.0	166726.00	775335.5	2267408.25	2.184247e+07
population	226.0	3.495521e+07	1.390338e+08	805.0	560512.50	5800569.5	21872837.75	1.439324e+09

Potential Issues:

```
# Missing values - count and percentage
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(1)
pd.DataFrame({'missing_count': missing, 'missing_pct': missing_pct}).query('missing_count > 0').sort_values('missing_pct', ascending=False)
```

	missing_count	missing_pct
serious_or_critical	81	35.8
active_cases	22	9.7
total_recovered	22	9.7
total_tests_per_1m_population	14	6.2
total_tests	14	6.2
total_deaths	8	3.5
total_deaths_per_1m_population	8	3.5

```
# Duplicate rows, and duplicate countries specifically (more meaningful than a full-row duplicate check)
print("Fully duplicated rows:", df.duplicated().sum())
print("Duplicate country names:", df['country'].duplicated().sum())
```

```
# Consistency checks - values that shouldn't logically happen
print("Negative values in numeric columns:")
num_cols = df.select_dtypes(include=np.number).columns
print((df[num_cols] < 0).sum()[lambda x: x > 0])

print("\nRecovered + Active > Confirmed (would be impossible):")
check = df['total_recovered'].fillna(0) + df['active_cases'].fillna(0) > df['total_confirmed'] * 1.05
print(check.sum(), "rows flagged")
```

```
Negative values in numeric columns:
Series([], dtype: int64)

Recovered + Active > Confirmed (would be impossible):
0 rows flagged
```

- **Missing Values:** Some columns contain missing values because not every country reported all statistics.
- **Duplicate Values:** The dataset was checked for duplicate rows and duplicate country names, no data found.
- **Data Consistency:** Logical checks were performed. Recovered Cases + Active Cases should not be much greater than Total Confirmed Cases. Negative values were also checked because they are not valid for COVID-19 statistics.

Data Cleaning

Cleaning the data improves the quality of the analysis.

Missing Values: Missing values were kept as NaN instead of replacing them. Replacing missing values with guessed numbers could introduce incorrect information. Most pandas functions automatically ignore missing values.

Duplicate Check: Duplicate rows and duplicate countries were checked. Duplicates may produce misleading analysis.

Consistency Check: Impossible values such as negative cases or inconsistent totals were checked. This ensures the dataset is logically correct before analysis.

```
df_clean = df.copy()

before = len(df_clean)
df_clean = df_clean.drop_duplicates()
print(f"Dropped {before - len(df_clean)} duplicate rows")

# Standardize text columns
df_clean['country'] = df_clean['country'].str.strip()
df_clean['continent'] = df_clean['continent'].str.strip()

print(df_clean['continent'].unique())
```

✓ 0.0s

Dropped 0 duplicate rows
<StringArray>
['Asia', 'Europe', 'Africa',
 'North America', 'South America', 'Australia/Oceania']
Length: 6, dtype: str

```
# Missing values - count and percentage
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(1)
pd.DataFrame({'missing_count': missing, 'missing_pct': missing_pct}).query('missing_count > 0').so
```

✓ 0.0s

	missing_count	missing_pct
serious_or_critical	81	35.8
active_cases	22	9.7
total_recovered	22	9.7
total_tests_per_1m_population	14	6.2
total_tests	14	6.2
total_deaths	8	3.5
total_deaths_per_1m_population	8	3.5

Issue	Technique	Why
serious_or_critical missing (~36%)	Left as missing (not imputed)	Too much missing to safely guess; filling with 0 would falsely say 'no critical cases' for countries that simply didn't report. We'll exclude it from analyses that need complete data, not fake a number.
total_recovered, active_cases missing (~10%)	Left as missing, excluded per-analysis	Missing here usually means "stopped tracking," not "zero"
total_tests, total_tests_per_1m_population missing (~6%)	Left as missing/NaN	Same logic imputing testing volume would be fabricating data
total_deaths missing (~4%)	Left as missing, dropped only for analyses that need it	Small enough to just exclude from death-focused stats via .dropna() per-analysis rather than deleting whole rows
Duplicates	Checked and dropped if found (0 found)	Duplicates would double-count a country
Text columns (country, continent)	Whitespace stripped and standardized	Prevents accidental grouping mistakes, like 'Asia ' vs 'Asia'.

In short: nothing was mass-imputed with mean or median. That technique only makes sense when data is missing completely at random here, the missing pattern itself tells a story (which countries have weaker reporting systems), so keeping it as NaN and being transparent about it in each analysis was the more honest choice.

Exploratory Data Analysis

Univariate: This analyzes one variable at a time.

Examples include:

- Histograms
- Distribution plots
- Bar charts

The analysis showed:

- COVID-19 cases vary greatly among countries.
- Death rates are not evenly distributed.
- Testing rates also differ significantly.

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sns.histplot(df_clean['total_cases_per_1m_population'], bins=30, kde=True, ax=axes[0,0], color='black')
axes[0,0].set_title('Distribution: Cases per 1M population')

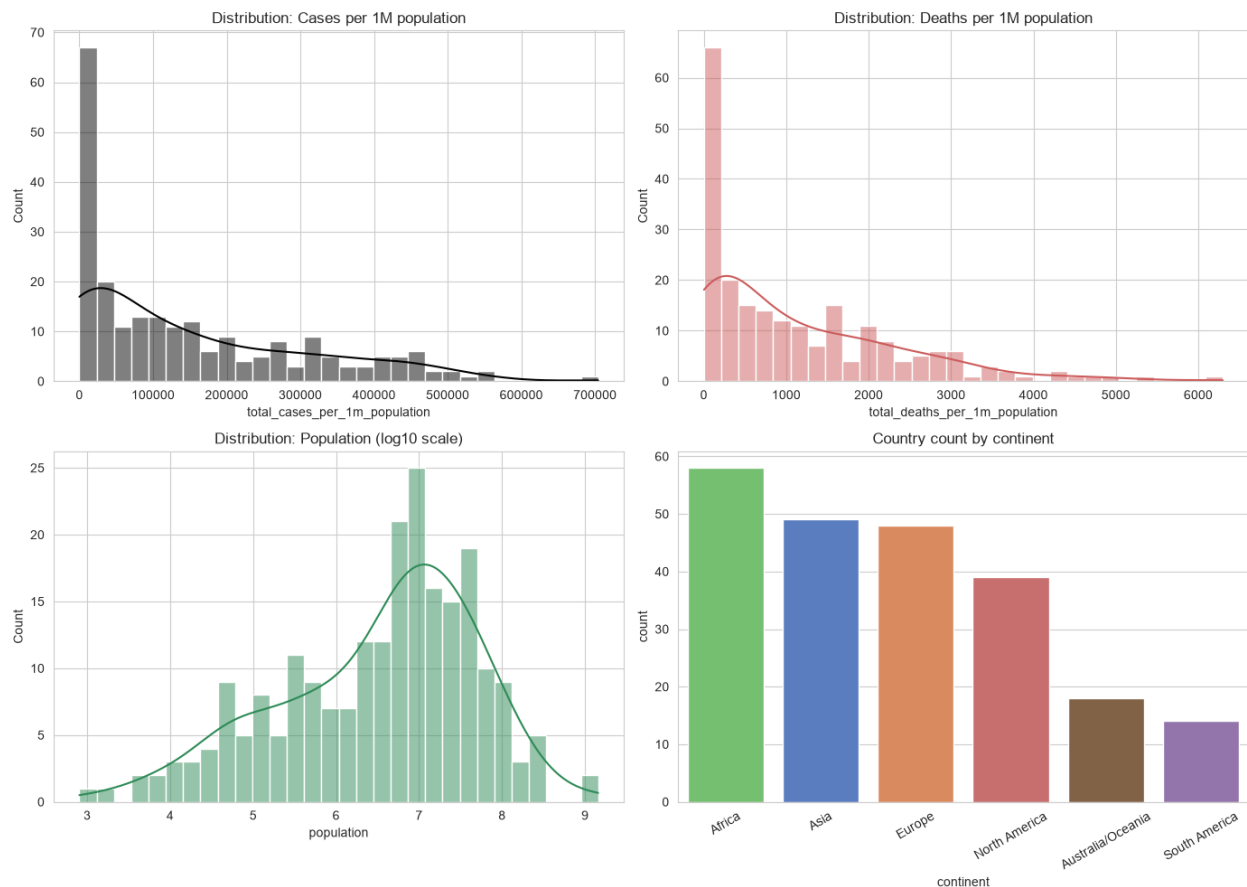
sns.histplot(df_clean['total_deaths_per_1m_population'].dropna(), bins=30, kde=True, ax=axes[0,1], color='indianred')
axes[0,1].set_title('Distribution: Deaths per 1M population')

sns.histplot(np.log10(df_clean['population']), bins=30, kde=True, ax=axes[1,0], color='seagreen')
axes[1,0].set_title('Distribution: Population (log10 scale)')

sns.countplot(data=df_clean, x='continent', hue='continent', order=df_clean['continent'].value_counts().index, ax=axes[1,1], palette='muted')
axes[1,1].set_title('Country count by continent')
axes[1,1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
```

✓ 0.4s



Key Understandings:

- Cases per 1M and deaths per 1M are both right-skewed most countries sit on the lower end, with a long tail of harder-hit countries.
- Population needed a log scale to be readable at all on a normal scale it would just be one tall bar near zero, since population ranges from about 800 people (tiny territories) to 1.4 billion (India/China).
- Africa has the most countries in the dataset (58), followed by Europe and Asia; Australia/Oceania has the fewest

Bivariate Analysis: This analyzes relationships between two variables.

- Population vs Confirmed Cases: Scatter plots show that countries with larger populations generally have more confirmed cases.
- Testing vs Cases: Countries performing more tests usually reported more cases.
- Deaths per Million by Continent: Boxplots compare death rates among continents.
- Cases per Million by Continent: Violin plots show how case distributions differ between continents.
- Cross-tabulation: Countries were grouped into:
 - Low case rate
 - Medium case rate
 - High case rate

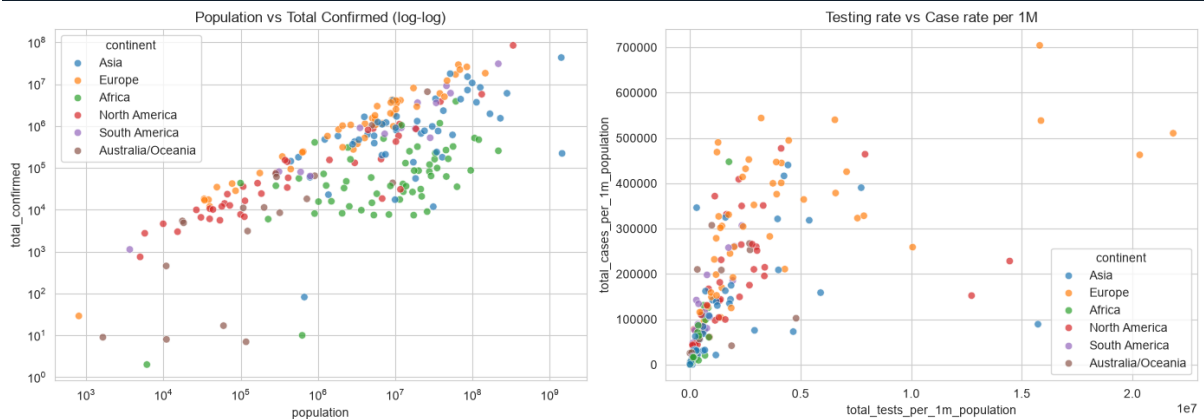

```
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

sns.scatterplot(data=df_clean, x='population', y='total_confirmed', hue='continent', ax=ax[0], alpha=0.7)
ax[0].set_xscale('log'); ax[0].set_yscale('log')
ax[0].set_title('Population vs Total Confirmed (log-log)')

sns.scatterplot(data=df_clean, x='total_tests_per_1m_population', y='total_cases_per_1m_population', hue='continent', ax=ax[1], alpha=0.7)
ax[1].set_title('Testing rate vs Case rate per 1M')

plt.tight_layout()
plt.show()
```

✓ 0.6s



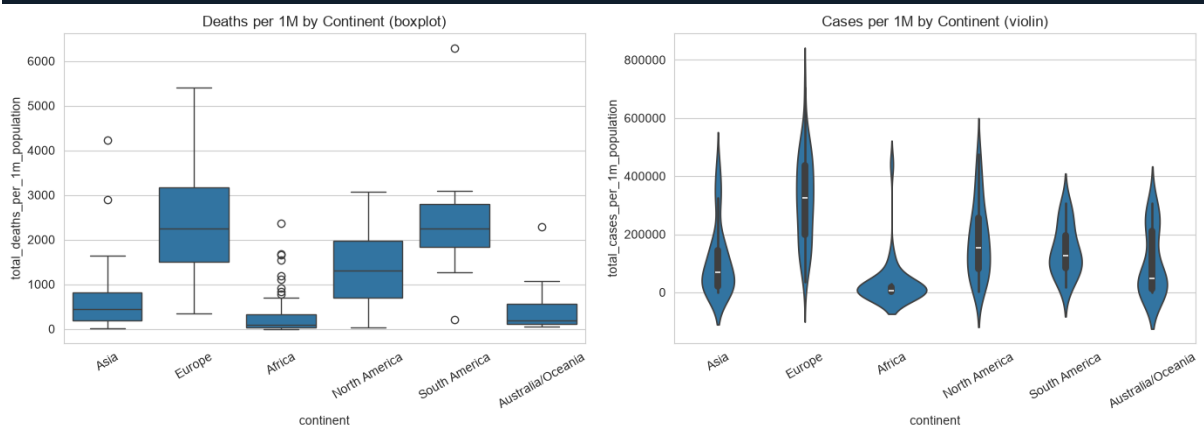
```
fig, ax = plt.subplots(1, 2, figsize=(14, 5))

sns.boxplot(data=df_clean, x='continent', y='total_deaths_per_1m_population', ax=ax[0])
ax[0].set_title('Deaths per 1M by Continent (boxplot)')
ax[0].tick_params(axis='x', rotation=30)

sns.violinplot(data=df_clean, x='continent', y='total_cases_per_1m_population', ax=ax[1])
ax[1].set_title('Cases per 1M by Continent (violin)')
ax[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.show()
```

✓ 0.2s



Key Understandings:

- Population vs. confirmed cases follows a roughly straight line on the log-log plot bigger countries tend to have proportionally more cases, which is expected.
- Testing rate vs. case rate shows a positive relationship (correlation ≈ 0.61). This is a classic detection-bias pattern: countries that test more per person tend to record more cases, not necessarily because the outbreak was worse, but because more infections

were actually caught. This matters it means 'more reported cases' isn't automatically 'worse outbreak.'

```
df_clean['case_rate_tier'] = pd.qcut(df_clean['total_cases_per_1m_population'], q=3, labels=['Low', 'Medium', 'High'])
pd.crosstab(df_clean['continent'], df_clean['case_rate_tier'])
```

✓ 0.0s

case_rate_tier	Low	Medium	High
continent			
Africa	47	9	2
Asia	17	24	8
Australia/Oceania	8	4	6
Europe	0	11	37
North America	3	19	17
South America	1	8	5

Multivariate Analysis: Multiple variables were analyzed together.

Pair Plot: The pair plot compares relationships among:

- Cases per million
- Deaths per million
- Tests per million

Different continents were shown using different colors.

```
num_cols_corr = ['total_confirmed', 'total_deaths', 'total_recovered', 'active_cases',
'serious_or_critical', 'total_cases_per_1m_population', 'total_deaths_per_1m_population',
'total_tests', 'total_tests_per_1m_population', 'population']

corr = df_clean[num_cols_corr].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, square=True, linewidths=0.5)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.show()
```

✓ 0.4s Python



Key Understandings:

- total_confirmed, total_deaths, total_recovered, and total_tests are all strongly correlated with each other (0.85–1.00) that's expected, since they're all 'raw counts' that scale with country size.
- The per-1M columns correlate much weaker with population (around -0.08 to -0.13) confirming they really do remove the size effect, which is exactly why they were used for the fair comparisons above.
- As always with this kind of aggregate country data: correlation is not causation. 'Moves together with' is the right phrase, not 'causes.'

```
subset_cols = ['total_cases_per_1m_population', 'total_deaths_per_1m_population', 'total_tests_per_1m_population', 'continent']
sns.pairplot(df_clean[subset_cols].dropna(), hue='continent', diag_kind='kde', height=2.2)
plt.suptitle('Pairwise relationships: cases, deaths, testing (per 1M)', y=1.02)
plt.show()
```

0.8s



The analysis confirms what the earlier scatter plots showed cases and deaths per 1M rise together, and European/South American countries (lighter and darker warm tones) cluster toward the higher end of both.

Key Insights Summary

- Scale distortion: Raw counts like total_confirmed and total_deaths are driven mostly by how big a country's population is, not by how badly it was hit. The per-1M columns tell the real story that's why this report leans on them throughout.
- Reporting gaps aren't random: Missing data clusters in serious_or_critical (36%) and total_tests (6%), and tends to affect countries with weaker testing/reporting infrastructure. This is a limitation worth flagging, not something to silently patch over.
- Europe and South America were hit hardest by the numbers: both have median death rates per 1M around 2,250 more than double North America's and over ten times Africa's median.
- Testing and case rate move together (correlation ≈ 0.61): countries that tested more per person tended to report more cases partly a real signal, partly a detection-bias effect.
- The five worst-hit countries by deaths per 1M were Peru (6,297), Bulgaria (5,407), Bosnia and Herzegovina (4,866), Hungary (4,820), and Macedonia (4,459) four of the five are in Europe.

The End